파이썬 완전 백과사전 2권 · 머신러닝 기초 교안

1일차

규칙을 짜지 않고 데이터로 배우기

1~5차시 · 책 1장 ~ 2장 2.6절

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/01-what-is-ml, 02-data-preparation

오늘의 시간표

차시	주제	책 범위
1	머신러닝이란: 데이터로 배우는 프로그램	1장 들어가며 ~ 1.3
2	첫 모델: 최근접 이웃	1.4 ~ 1.6
3	일반화와 과적합, 평가 맛보기	1.7 ~ 1.11, 1.13
4	데이터 살펴보기, 나누기, 누수	2장 들어가며 ~ 2.3
5	결측치, 범주형 인코딩, 스케일링	2.4 ~ 2.6

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분

준비: examples/vol2 폴더에서 uv sync · 1장 예제 25개는 모두 합쳐 20초 안쪽으로 돈다

1차시

01

머신러닝이란: 데이터로 배우는 프로그램

책 1장 들어가며 ~ 1.3절

학습 목표

- 규칙을 짜는 프로그램과 데이터로 배우는 프로그램의 차이를 말한다
- 샘플, 특성, 레이블로 데이터 표를 읽는다
- 지도(분류·회귀), 비지도, 강화 학습을 예로 가른다

사람이 금지어를 적으면

```
def 스팸인가(제목: str) -> bool:
    금지어 = ["무료", "당첨", "대출"]
    for 단어 in 금지어:
        if 단어 in 제목:
            return True
    return False

for 제목 in 메일들:
    판정 = "스팸" if 스팸인가(제목) else "정상"
    print(f"[{판정}] {제목}")
```

```
[스팸] 축하합니다 경품 당첨 안내
[스팸] 무료 체험 쿠폰이 도착했어요
[정상] 내일 회의 자료 공유드립니다
[정상] 무.료 상담 지금 바로 전화 주세요
[정상] 저금리 대환 상품 한정 안내
[스팸] 도서관 책 대출 기한이 내일까지예요
```

- "무.료"로 바꾸자 그냥 통과
- 목록에 없는 "저금리 대환"도 통과
- 멀쩡한 도서관 **대출** 안내는 스팸으로

표시만 하고, 규칙은 데이터에서 찾게 하면

```
스팸_단어 = Counter()
정상_단어 = Counter()
for 제목, 표시 in 학습_메일:  # 표시가 붙은 12통
    if 표시 == "스팸":
        스팸_단어.update(제목.split())
    else:
        정상_단어.update(제목.split())

def 스팸_점수(제목: str) -> int:
    return sum(스팸_단어[w] - 정상_단어[w]
               for w in 제목.split())
```

```
[스팸] 점수 +5  무.료 체험 지금 신청
[스팸] 점수 +4  저금리 대환 특가 안내
[정상] 점수 -1  도서관 대출 연장 안내
[정상] 점수 -4  다음 주 회의 자료
```

- 금지어 목록이 없다
- 점수가 0보다 크면 스팸
- 새 수법이 나오면 규칙 대신 **예를 더한다**

1.1 입력과 출력의 자리가 바뀝니다

	전통적인 프로그래밍	머신러닝
사람이 주는 것	규칙(코드) + 데이터	데이터 + 정답
컴퓨터가 내놓는 것	답	규칙(모델)
규칙을 바꾸려면	코드를 고친다	데이터를 바꾸고 다시 학습
잘 맞는 문제	세금 계산, 정렬, 회원 가입	스팸 거르기, 사진 분류, 추천

두 질문에 모두 "그렇다"일 때 꺼내는 도구: **규칙을 사람이 적기 어려운가? 예가 될 데이터가 충분한가?**

1.2 데이터는 표 하나와 정답 목록 하나

```
X 모양: (150, 4)  y 모양: (150,)
레이블 이름: ['setosa', 'versicolor', 'virginica']
  0번 꽃: 특성 [5.1 3.5 1.4 0.2] → 레이블 0 (setosa)
 50번 꽃: 특성 [7.  3.2 4.7 1.4] → 레이블 1 (versicolor)
100번 꽃: 특성 [6.3 3.3 6.  2.5] → 레이블 2 (virginica)
레이블별 개수: [50 50 50]
```

낱말	뜻	붓꽃에서는
샘플	데이터 한 건, 표의 한 행	꽃 한 송이
특성	샘플을 설명하는 항목, 표의 한 열	꽃받침·꽃잎 길이와 너비
레이블	맞히고 싶은 정답	품종(0, 1, 2)

1차시 · 1.2 샘플, 특성, 레이블 · 예제 01-what-is-ml/b03_iris_look.py (출력 일부)

1.3 무엇으로 배우나: 세 갈래

지도 학습

정답이 달린 문제집으로 배운다

분류: 답을 고른다
붓꽃 레이블 [0 1 2]

회귀: 답을 잰다
당뇨 진행도 25~346, 214가지 값

비지도 학습

정답 없이 비슷한 것끼리 묶는다

품종을 숨기고 KMeans로 세 묶음: 한 묶음에 setosa 50송이가 정확히 모였다

강화 학습

해 보고 보상으로 배운다

뽑기 기계 셋, 1000번: 가장 좋은 기계를 찾아 885번 누르고 672번 당첨

직접 실행 10분

1. 01-what-is-ml 폴더에서 b01_rule_spam.py와 b02_learn_spam.py를 차례로 실행하고, 두 결과에서 무엇이 달라졌는지 적는다
2. b02의 학습_메일에 ("대출 한도 무료 조회", "스팸") 한 줄을 더하고 다시 실행한다. "도서관 대출 연장 안내"의 점수는 어떻게 바뀌나?
3. b04_kinds.py로 분류 레이블과 회귀 레이블이 어떻게 다른지, b05_unsupervised.py로 정답 없이 묶은 결과를 출력으로 확인한다

```
cd examples/vol2/01-what-is-ml && uv run python b02_learn_spam.py
```

다음 중 회귀 문제는 무엇일까요?

Q

가 받은 메일이 스팸인지 맞히기

나 내일 매장 방문자 수 맞히기

다 사진 속 손글씨 숫자 0~9 맞히기

라 고객을 구매 패턴으로 묶기

1차시 정리

머신러닝은 규칙을 짜는 대신 데이터와 정답을 주고, 규칙은 프로그램이 배우게 하는 방법이다

데이터는 **샘플 × 특성** 표 X 하나와 **레이블** 목록 y 하나로 정리된다

정답으로 배우면 **지도**(고르면 분류, 재면 회귀), 정답 없이 묶으면 **비지도**, 보상으로 배우면 **강화**

다음 차시: 모델을 직접 만든다 · 최근접 이웃 분류

2차시

02

첫 모델: 최근접 이웃

책 1.4 ~ 1.6절

학습 목표

- 모델, 파라미터, 하이퍼파라미터를 구별한다
- 최근접 이웃을 NumPy로 직접 짜고 scikit-learn과 결과를 맞춘다
- fit · predict · score 약속으로 분류와 회귀를 돌린다

1.4 모델은 빈칸이 있는 함수

```
X, y = load_iris(return_X_y=True)
고름 = y > 0
꽃잎_길이 = X[고름, 2]
정답 = y[고름]

최고_기준, 최고_정확도 = None, 0.0
for 기준 in np.unique(꽃잎_길이):
    예측 = np.where(꽃잎_길이 > 기준, 2, 1)
    정확도 = (예측 == 정답).mean()
    if 정확도 > 최고_정확도:
        최고_기준, 최고_정확도 = 기준, 정확도
```

```
시험한 기준 수: 34
배운 규칙: 꽃잎 길이 > 4.7 이면 virginica
학습 데이터에서 정확도: 0.93
```

1. **모양**을 정한다: 길이 > 기준이면 virginica
2. **잣대**를 정한다: 정확도
3. 잣대가 가장 좋은 **빈칸 값**을 찾는다: 4.7

파라미터는 학습이 찾고
하이퍼파라미터는 사람이 고른다

2차시 · 1.4 모델, 학습, 예측 · 예제 01-what-is-ml/b07_one_rule.py (핵심 부분)

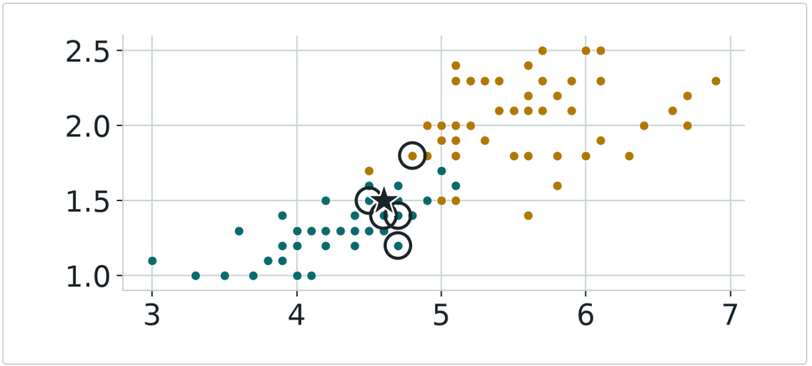
1.5 끼리끼리: 가까운 이웃의 다수결

```
새_꽃 = np.array([6.0, 2.9, 4.6, 1.5])

차이 = X - 새_꽃
거리 = np.sqrt((차이 ** 2).sum(axis=1))
print("거리 모양:", 거리.shape)

가까운_순 = np.argsort(거리)[:5]
for 번호 in 가까운_순:
    print(f"{번호:3d}번 꽃  거리 {거리[번호]:.3f}  레이블 {y[번호]}")
print("다섯 이웃의 표:", np.bincount(y[가까운_순], minlength=3))
```

```
거리 모양: (150,)
 78번 꽃  거리 0.100  레이블 1
 91번 꽃  거리 0.173  레이블 1
 63번 꽃  거리 0.173  레이블 1
 73번 꽃  거리 0.346  레이블 1
138번 꽃  거리 0.374  레이블 2
다섯 이웃의 표: [0 4 1]
```



★ 새 꽃 · ◯ 이웃 · versicolor · virginica

2차시 · 1.5 거리 재기 · 예제 01-what-is-ml/b08_distance.py (핵심 부분, 그림은 실행 결과를 꽃잎 길이·너비로 그린 것)

1.5 시험 문제는 따로 숨겨 둔다

```
rng = np.random.default_rng(42)
섞은_번호 = rng.permutation(len(X))    # 0~149를 무작위 순서로
테스트_번호 = 섞은_번호[:30]            # 30송이는 시험용으로 숨겨 두고
학습_번호 = 섞은_번호[30:]              # 나머지 120송이로 배운다

X_학습, y_학습 = X[학습_번호], y[학습_번호]
X_테스트, y_테스트 = X[테스트_번호], y[테스트_번호]
```

```
학습: (120, 4) 품종별 [40 41 39]
테스트: (30, 4) 품종별 [10  9 11]
```

- 배운 데이터로 시험하면 점수가 부풀려진다
- 섞지 않고 자르면 테스트에 setosa만 들어간다
- 씨앗 42를 고정하면 늘 같은 30송이

1.5 NumPy로 직접 만드는 최근접 이웃

```
def 최근접_이웃_예측(X_학습: np.ndarray, y_학습: np.ndarray,
                     X_새: np.ndarray, k: int) -> np.ndarray:
    # (새 개수, 1, 특성) - (1, 학습 개수, 특성) → (새 개수, 학습 개수, 특성)
    차이 = X_새[:, np.newaxis, :] - X_학습[np.newaxis, :, :]
    거리 = np.sqrt((차이 ** 2).sum(axis=2))           # (새 개수, 학습 개수)
    이웃 = np.argsort(거리, axis=1, kind="stable")[:, :k]  # 가까운 k개의 번호
    이웃_레이블 = y_학습[이웃]                          # (새 개수, k)
    return np.array([np.bincount(줄).argmax() for 줄 in 이웃_레이블])

예측 = 최근접_이웃_예측(X_학습, y_학습, X_테스트, k=5)
```

```
예측: [2 1 2 0 2 2 1 2 2 0]
정답: [1 1 2 0 2 2 1 2 2 0]
테스트 정확도: 0.967 (29/30)
```

- (30, 1, 4) − (1, 120, 4) → **3600쌍**의 거리를 한 번에
- 틀린 하나: versicolor를 virginica로

2차시 · 1.5 NumPy로 직접 만들기 · 예제 01-what-is-ml/b10_knn_numpy.py (핵심 부분)

1.5 scikit-learn 세 줄로, 결과는 똑같이

```
모델 = KNeighborsClassifier(n_neighbors=5)   # 1) 모델 고르기
모델.fit(X_학습, y_학습)                      # 2) 학습
예측 = 모델.predict(X_테스트)                 # 3) 예측
print(f"scikit-learn 정확도: {모델.score(X_테스트, y_테스트):.3f}")

직접 = 최근접_이웃_예측(X_학습, y_학습, X_테스트, k=5)
print("직접 짠 것과 예측이 모두 같은가:", np.array_equal(예측, 직접))
```

```
scikit-learn 정확도: 0.967
직접 짠 것과 예측이 모두 같은가: True
```

- 안에서 하는 일은 같다: 거리 → k개 → 투표
- 더 들어 있는 것: 빠른 이웃 찾기, 여러 거리, 오류 검사
- 원리는 직접 짜 보고, 실무는 검증된 라이브러리로

1.5 얼마나 확신하나: predict_proba

```
붓꽃 = load_iris()
모델 = KNeighborsClassifier(n_neighbors=5).fit(붓꽃.data, 붓꽃.target)

새_꽃들 = np.array([[5.0, 3.4, 1.5, 0.2],    # 예제용으로 지어낸 꽃 세 송이
                    [6.0, 2.9, 4.6, 1.5],
                    [6.3, 2.8, 5.0, 1.7]])
확률 = 모델.predict_proba(새_꽃들)            # 이웃 다섯 중 품종별 비율
```

```
[5.  3.4 1.5 0.2] → [1. 0. 0.]  setosa
[6.  2.9 4.6 1.5] → [0.  0.8 0.2]  versicolor
[6.3 2.8 5.  1.7] → [0.  0.2 0.8]  virginica
```

- 5 대 0과 4 대 1은 같은 "정답"이라도 확신이 다르다
- 확신이 낮은 샘플만 사람이 다시 확인
- "80% 이상일 때만 자동 처리" 같은 규칙

2차시 · 1.5 얼마나 확신하나요? · 예제 01-what-is-ml/b24_proba.py (핵심 부분)

1.5 fit과 predict라는 약속

```
class 최근접이웃분류기:
    def __init__(self, n_neighbors: int = 5) -> None:
        self.n_neighbors = n_neighbors

    def fit(self, X: np.ndarray, y: np.ndarray) -> "최근접이웃분류기":
        self.X_, self.y_ = X.copy(), y.copy()
        return self

for 모델 in [최근접이웃분류기(n_neighbors=3), KNeighborsClassifier(n_neighbors=3)]:
    모델.fit(X_학습, y_학습)
    print(f"{type(모델).__name__:22s} 정확도 {모델.score(X_테스트, y_테스트):.3f}")
```

```
최근접이웃분류기                    정확도 0.967
KNeighborsClassifier    정확도 0.967
```

- 이름이 같은 메서드면 같은 반복문에: **덕 타이핑**
- 학습은 복사뿐, 계산은 예측 때: "게으른 학습"

2차시 · 1.5 fit과 predict라는 약속 · 예제 01-what-is-ml/b12_my_class.py (핵심 부분: predict, score는 생략)

1.6 숫자를 맞히는 이웃: 회귀

```
X, y = load_diabetes(return_X_y=True)
X_학습, X_테스트, y_학습, y_테스트 = train_test_split(X, y, test_size=0.25, random_state=0)
# 비교 기준: 누구에게나 학습 데이터의 평균을 답한다
평균_답 = np.full(len(y_테스트), y_학습.mean())

# 최근접 이웃 회귀: 가까운 k명의 레이블 평균을 답한다
for k in [1, 10, 30]:
    예측 = KNeighborsRegressor(n_neighbors=k).fit(X_학습, y_학습).predict(X_테스트)
```

```
평균으로 답하기: 평균 오차 58.3
이웃  1명의 평균: 평균 오차 64.9
이웃 10명의 평균: 평균 오차 46.5
이웃 30명의 평균: 평균 오차 43.1
테스트 환자 셋의 정답: [321. 215. 127.]  k=30 예측: [228.  209.6 146. ]
```

- 투표 대신 **평균**
- 회귀는 얼마나 **빗나갔나**를 잰다
- 비교 기준부터 잰다
- k=1은 기준보다 못하다

2차시 · 1.6 숫자를 맞히는 이웃: 회귀 · 예제 01-what-is-ml/b21_regression.py (핵심 부분)

직접 실행 10분

1. 01-what-is-ml 폴더에서 b10_knn_numpy.py와 b11_sklearn.py를 실행해 0.967과 True를 확인한다
2. b11의 n_neighbors=5와 k=5를 둘 다 1로, 다시 둘 다 15로 바꿔 실행한다. 정확도는 어떻게 될까?
3. b08의 새_꽃을 [6.3, 2.8, 5.0, 1.7]로 바꾸면 다섯 이웃의 표는? predict_proba 결과와 맞춰 본다
4. 더 할 사람: b21의 k 목록에 100과 300을 더해 본다

```
cd examples/vol2/01-what-is-ml && uv run python b11_sklearn.py
```

다음 중 학습이 데이터에서 찾아내는 값, 파라미터는 무엇일까요?

Q

가 최근접 이웃에서 볼 이웃의 수 k

나 "꽃잎 길이 > 4.7이면 virginica"의 4.7

다 꽃잎 길이 특성 하나만 쓰기로 한 결정

라 테스트용으로 숨겨 둘 꽃의 수 30

2차시 정리

모델은 빈칸(파라미터)이 있는 함수, **학습**은 잣대가 좋아지는 빈칸 값 찾기, **하이퍼파라미터**는 사람이 고른다

최근접 이웃: 거리를 재고 가까운 k개로 투표(분류)하거나 평균(회귀). 직접 짠 것과 scikit-learn이 같은 답

시험용 데이터는 숨겨 두고, **fit**으로 배우고 **predict**로 답하고 **score**로 채점한다

다음 차시: 외우기와 이해하기 · 일반화, 과적합, 평가 맛보기

3차시

03

일반화와 과적합, 평가 맛보기

책 1.7 ~ 1.11절, 1.13 실습

학습 목표

- 학습 점수와 테스트 점수를 나란히 보고 과적합과 과소적합을 가린다
- 거리를 쓰는 모델에 눈금 맞추기(표준화)가 왜 필요한지 말한다
- 비교 기준과 혼동 행렬로 점수를 읽고, 작업 흐름 다섯 단계를 말한다

1.7 기출문제만 외운 학생

```
# 외우기 모델: 특성 네 개를 통째로 열쇠로 삼아 정답을 적어 둔다
암기장 = {tuple(특성): 정답 for 특성, 정답 in zip(X_학습, y_학습)}

def 외워서_답하기(특성: np.ndarray) -> int | None:
    return 암기장.get(tuple(특성))      # 처음 보는 꽃이면 None
```

```
학습 데이터: 120/120 맞힘
테스트 데이터: 0/30 맞힘, 모른다(None): 30송이
```

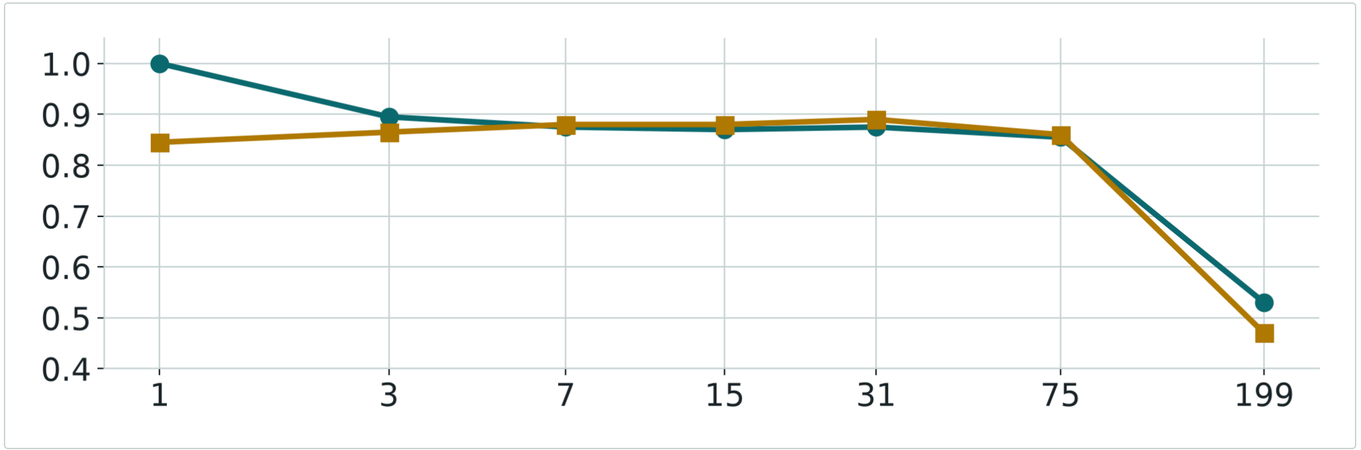
학습 정확도 100%가
쓸모없는 모델의 성적표일 수도 있다

일반화: 학습 데이터가 아니라 **처음 보는 데이터**를 잘 맞히는 능력

1.7 k가 작으면 외우고, 너무 크면 뭉갠다

```
# 초승달 두 개가 맞물린 모양의 점들 (예제용으로 만든 데이터), 잡음을 섞었다
X, y = make_moons(n_samples=400, noise=0.35, random_state=0)
```

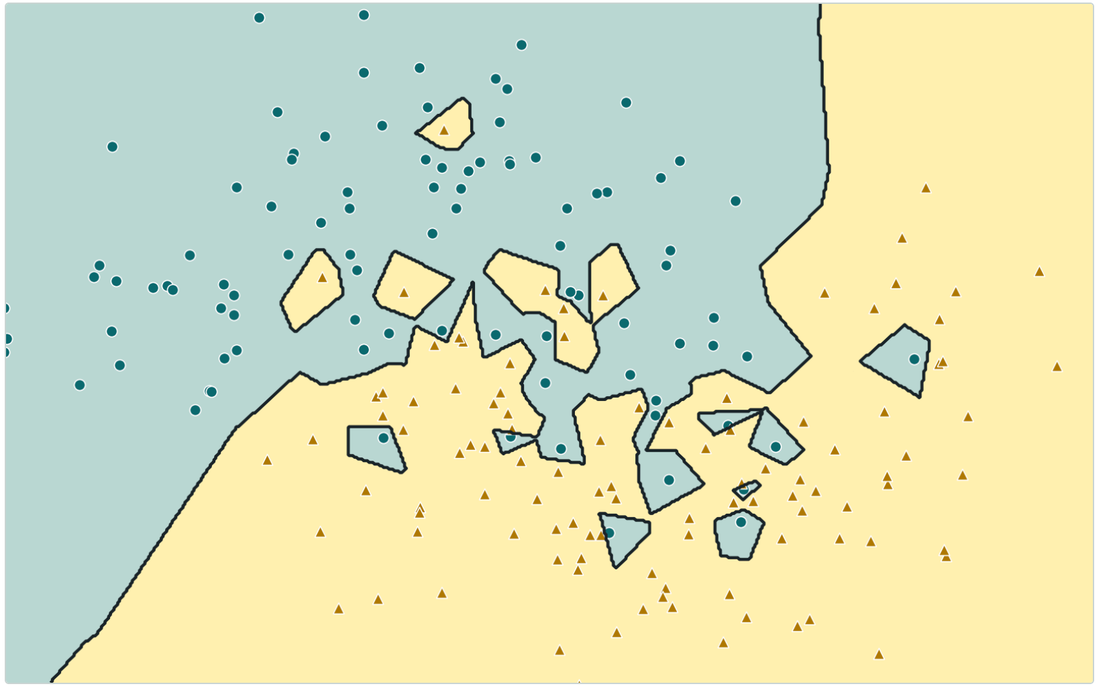
k	학습 정확도	테스트 정확도
1	**1.000**	0.845
3	0.895	0.865
7	0.875	0.880
15	0.870	0.880
31	0.875	**0.890**
75	0.855	0.860
199	0.530	0.470



학습만 높으면 **과적합**(k가 너무 작음), 둘 다 낮으면 **과소적합**(k가 너무 큼), 둘 다 높고 비슷하면 **알맞음**

1.7 경계선을 그려 보면

k=1: 점 하나하나에 섬이 생긴다 · 테스트 0.845



k=31: 매끄러운 곡선 하나 · 테스트 0.890

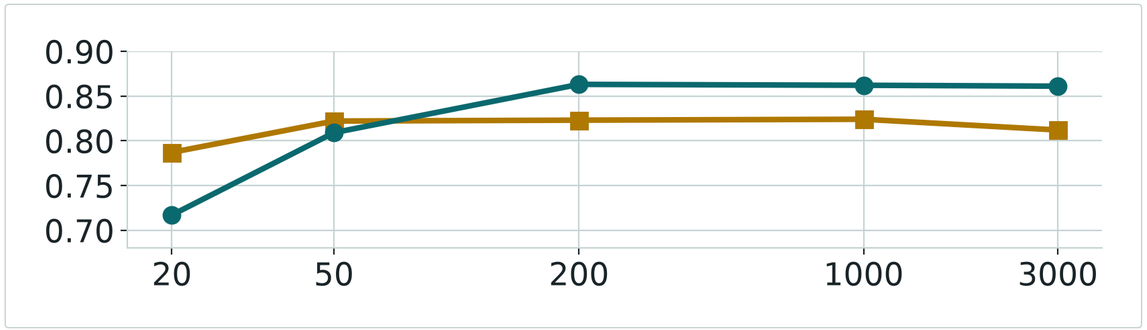


과적합한 모델은 경계가 복잡하고, 알맞은 모델은 경계가 **데이터의 큰 흐름**을 따른다 (청록 ● 0번 달, 노랑 ▲ 1번 달, 학습 점 200개)

3차시 · 1.7 경계선을 그려 보면 · 책의 b23_boundary.py와 같은 데이터·모델을 색칠한 그림

1.7 데이터가 많을수록 외우기 어렵다

학습 개수	k=1 테스트	k=15 테스트
20	0.787	0.717
50	0.822	0.809
200	0.823	**0.863**
1000	0.824	0.862
3000	0.812	0.861



● k=15 · ■ k=1 · 가로는 학습 개수

- 데이터가 늘면 성능이 오른다: 0.717 → 0.863
- 200개를 넘으면 0.86 언저리: **데이터 자체의 한계**
- k=1은 3000개여도 0.81~0.82: 잡음까지 외우는 버릇은 안 고쳐진다

데이터를 늘리는 것과 모델을 알맞게 고르는 것, **둘 다** 필요하다

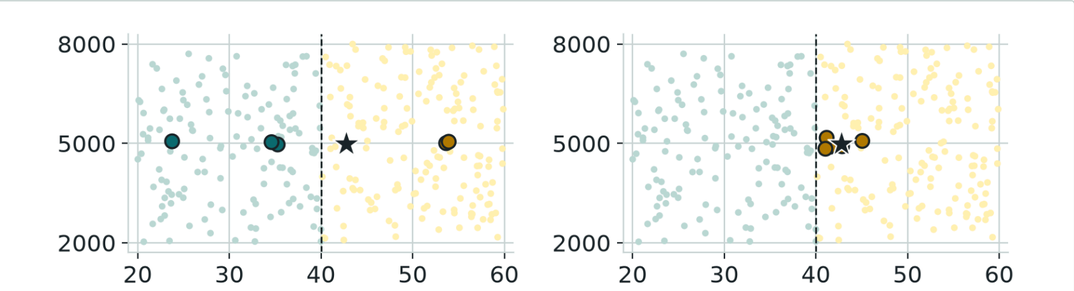
1.8 특성의 눈금 맞추기

```
나이 = rng.uniform(20, 60, 400)
연소득 = rng.uniform(2000, 8000, 400)
y = (나이 > 40).astype(int)          # 구매 여부는 나이로만 정해진다고 하자

# 학습 데이터의 평균과 표준편차로 두 특성의 눈금을 맞춘다 (표준화)
평균, 표준편차 = X_학습.mean(axis=0), X_학습.std(axis=0)
표준화 = KNeighborsClassifier(n_neighbors=5).fit((X_학습 - 평균) / 표준편차, y_학습)
```

```
그대로:     정확도 0.510
표준화 후: 정확도 0.980
특성별 표준편차(나이, 연소득): [  11.9 1643.5]
```

- 나이 20세 차이는 연소득 수백만 원 차이에 묻힌다
- 평균과 표준편차는 **학습 데이터에서만**



왼쪽 그대로 · 오른쪽 표준화 · ★ 테스트 고객 · 진한 점 = 가까운 다섯

1.9 얼마나 잘하나: 평가 맛보기

비교 기준과 나란히

```
늘 1번이라고 답하면: 0.300
최근접 이웃(k=5):       0.967
```

가장 흔한 품종만 찍는 "모델"보다 훨씬 높다

어디서 틀렸나

```
혼동 행렬 (행: 정답, 열: 예측)
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
```

대각선은 맞힘. 틀린 1송이는 versicolor → virginica

정확도의 함정

```
늘 '정상'이라고 답하는 모델의 정확도: 0.990, 찾아낸 사기: 0건
```

거래 1000건 중 사기 10건. 드문 쪽이 중요하면 다른 잣대가 필요하다

1.10 머신러닝 작업의 흐름: 돌고 돈다

단계	하는 일	스팸 거르기라면
1. 문제 정의	무엇을 맞힐지, 무엇이 성공인지	정상 메일을 스팸으로 보내는 일은 1000통에 1통 이하
2. 데이터	모으고, 레이블 붙이고, 정리하고, 나눈다	사용자가 누른 "스팸 신고" 모으기
3. 모델	모델을 고르고 학습한다	단어 기반 모델에서 신경망까지
4. 평가	떼어 둔 데이터로 재고, 틀린 사례를 본다	놓친 스팸, 잘못 막은 메일 살펴보기
5. 배포와 관찰	서비스에 붙이고 성능을 지켜본다	메일 서버에 붙이고 신고율 지켜보기

```
모델에 바로 묻기: virginica
창구로 묻기 1: setosa
창구로 묻기 2: 모르겠어요 (배운 적 없는 크기)
```

꽃잎 100cm에도 모델은 답한다.
배포는 모델 앞에 **창구**를 세우는 것부터

3차시 · 1.10 머신러닝 작업의 흐름 · 예제 01-what-is-ml/b25_service.py (출력)

1.11 딥러닝은 무엇이 다른가: 특성은 누가 고르나

사람이 고른 특성 vs 픽셀 그대로

```
손으로 만든 특성 4개: 정확도 0.635
픽셀 64개 그대로: 정확도 0.970
```

잉크 양, 위·왼쪽·가운데 비율로는 1과 7이 잘 안 갈린다

픽셀 그대로면 충분할까

```
오른쪽으로 0칸 밀었을 때 정확도: 0.981
오른쪽으로 1칸 밀었을 때 정확도: 0.567
오른쪽으로 2칸 밀었을 때 정확도: 0.124
```

같은 자리 픽셀만 비교하니 **모양**을 못 잰다

	고전적인 머신러닝	딥러닝
특성	사람이 설계한다	모델이 원본 데이터에서 배운다
잘 맞는 데이터	표 형태(고객, 거래, 센서 수치)	이미지, 소리, 글

1.13 실습: 와인 생산자 맞히기

1. 모양과 특성 범위 살펴보기
2. 학습 106 · 검증 36 · 테스트 36으로 나누기
3. 학습 데이터 기준으로 표준화
4. **검증** 데이터로 표준화 여부 × k 고르기
5. **테스트**는 마지막에 한 번, scikit-learn과 맞춰 보기

```
모양: (178, 13)  생산자별: [59 71 48]
특성별 범위 예: proline 278.0 ~ 1680.0 / nonflavanoid_phenols 0.13 ~ 0.66
그대로 k=1:0.889 k=3:0.833 k=5:0.806 k=9:0.694 k=15:0.778
표준화 k=1:0.944 k=3:0.944 k=5:0.917 k=9:0.944 k=15:0.917
고른 설정: 표준화, k=9
테스트 정확도: 0.944 (34/36)
scikit-learn과 예측이 같은가: True
```

왜 테스트로 k를 고르면 안 될까? 가장 높은 테스트 점수는 "우연히 잘 맞은 설정"의 점수라 **부풀려진다**

직접 실행 10분

1. b22_practice.py의 씨앗 default_rng(7)을 0과 1로 바꿔 실행한다. 고른 설정과 테스트 정확도는 그대로일까?
2. b14_moons_overfit.py의 noise=0.35를 0.1로 바꾸면 k=1의 테스트 정확도는? 잡음이 적으면 외우기가 왜 덜 해로울까?
3. b16_scale.py의 정답을 y = (연소득 > 5000).astype(int)로 바꾸면 "그대로" 정확도는?

```
cd examples/vol2/01-what-is-ml && uv run python b22_practice.py
```

학습 정확도 1.000, 테스트 정확도 0.845. 이 모델은 어떤 상태일까요?

Q

가 과소적합: 공부를 안 했다

나 과적합: 답만 외웠다

다 알맞음: 원리를 이해했다

라 판단할 수 없다

3차시 정리

목표는 **처음 보는 데이터**의 점수. k가 작으면 과적합, 너무 크면 과소적합. 데이터가 많을수록 외우기 어렵다

거리를 재는 모델은 **눈금을 맞춘다**. 평균과 표준편차는 학습 데이터에서만 구한다

점수는 **비교 기준**과 함께, 틀린 곳은 혼동 행렬로. 고르기는 검증으로, 테스트는 마지막에 한 번

다음 차시: 2장 데이터 준비 · 살펴보기, 나누기, 누수

4차시

04

데이터 살펴보기, 나누기, 누수

책 2장 들어가며 ~ 2.3절

학습 목표

- 데이터를 받자마자 확인할 것들을 체크리스트로 점검한다
- 학습·검증·테스트로 나누는 이유를 말하고 직접 나눈다
- 누수가 점수를 얼마나 부풀리는지 보고, 막는 순서를 익힌다

장 봐 온 재료를 바로 냄비에 넣으면

```
고객 = pd.DataFrame({
    "나이": [34, 52, np.nan, 41, 29],
    "지역": ["서울", "부산", "서울", "대구", "부산"],
    "월요금": [55000, 89000, 32000, np.nan, 45000],
    "이탈": [0, 1, 0, 1, 0],
})
X = 고객.drop(columns="이탈")     # 문제(특성)
y = 고객["이탈"]                  # 정답(레이블)

모델 = LogisticRegression()
try:
    모델.fit(X, y)                  # 그대로 넣어 보면?
except ValueError as e:
    print("오류 1:", str(e).splitlines()[0])
```

```
오류 1: could not convert string to float: '서울'
오류 2: Input X contains NaN.
```

- 모델은 **숫자만** 계산한다
- 글자 열을 빼고 숫자만 넣으면 이번엔 **빈칸**에서 멈춘다(오류 2)
- 더 무서운 건 오류가 **안 나는** 문제들

4차시 · 2장 들어가며 · 예제 02-data-preparation/b01_raw_fail.py (핵심 부분: 오류 2를 낸 두 번째 try는 생략)

2.1 먼저 건강검진부터

```
모양: (10, 5)
자료형: {'회원번호': 'int64', '나이': 'float64', '지역': 'str', '가입일': 'str', '탈퇴': 'int64'}
빈칸 수: {'회원번호': 0, '나이': 1, '지역': 1, '가입일': 0, '탈퇴': 0}
중복 행: 1 개
지역 값: {'서울': 3, '부산': 2, '대구': 2, '서울시': 1, '서울 ': 1, nan: 1}
나이 범위: 0.0 ~ 999.0
정답 비율: {0: 0.7, 1: 0.3}
```

빈칸: 나이, 지역에 하나씩

숨은 빈칸: 0살, 999살

표기 흔들림: 서울, 서울시, "서울 "

중복 행: 107번이 두 번

자료형: 가입일이 글자(str)

정답 비율: 탈퇴 30%

2.1 데이터 살펴보기 체크리스트

확인할 것	코드	이런 게 보이면
크기	`df.shape`	생각보다 적거나 많으면 불러오기 의심
자료형	`df.dtypes`	숫자여야 할 열이 str이면 글자가 섞임
빈칸	`df.isna().sum()`	결측치 처리 (5차시)
숨은 빈칸	`df.describe()` 최솟값·최댓값	0, -1, 999, 9999 같은 수상한 값
범주 값	`df[열].value_counts()`	표기 흔들림, 오타, 너무 드문 값
중복	`df.duplicated().sum()`	같은 기록이 두 번 들어감
정답 비율	`y.value_counts(normalize=True)`	한쪽이 아주 적으면 불균형 (2일차)
너무 좋은 특성	정답과의 상관	누수 의심 (이번 차시)
시간 열	날짜 범위	시간 순서로 나눌지 결정 (2일차)

2.2 모의고사와 수능: 셋으로 나누기

나눈 데이터	비유	하는 일
학습 데이터	교과서와 문제집	모델이 보고 규칙을 배운다
검증 데이터	모의고사	어떤 모델·설정이 나은지 비교하고 고른다
테스트 데이터	**수능**	마지막에 딱 한 번, 진짜 실력을 잰다

```
# 1단계: 테스트용 20%를 먼저 떼어 금고에 넣는다
X_나머지, X_테스트, y_나머지, y_테스트 = train_test_split(
    X, y, test_size=0.2, random_state=42)
# 2단계: 남은 80% 가운데 25%(전체의 20%)를 검증용으로
X_학습, X_검증, y_학습, y_검증 = train_test_split(
    X_나머지, y_나머지, test_size=0.25, random_state=42)
```

```
학습/검증/테스트: 300 100 100
```

- 테스트를 **맨 먼저** 뗀다
- 돌려받는 순서: X 둘, y 둘
- 씨앗을 고정해야 같은 시험지

4차시 · 2.2 학습·검증·테스트로 나누기 · 예제 02-data-preparation/b03_split.py (핵심 부분)

2.2 시험지를 보고 고르면 생기는 일

```
# 정답이 동전 던지기로 정해진 데이터: 어떤 모델도 50%보다 잘할 수 없다
for 번호 in range(50):                                  # 후보 모델 50개
    열 = rng.choice(10, size=3, replace=False)          # 특성 3개를 골라 쓰는 모델
    k = int(rng.integers(1, 30))
    모델 = KNeighborsClassifier(n_neighbors=k).fit(X_학습[:, 열], y_학습)
    기록.append((모델.score(X_시험[:, 열], y_시험), 열, 모델))

시험점수, 열, 최고 = max(기록, key=lambda 행: 행[0])
```

61%

시험지를 보며 고른 최고 모델의 시험 점수

50.1%

같은 모델을 처음 보는 데이터 2만 개로 채점

4차시 · 2.2 시험지를 보고 고르면 생기는 일 · 예제 02-data-preparation/b04_peek.py (핵심 부분)

2.3 데이터 누수: 나누기 전에 특성을 고르면

```
X = rng.normal(size=(100, 5000))      # 100명 × 특성 5000개, 전부 의미 없는 난수
y = rng.integers(0, 2, size=100)      # 정답도 동전 던지기

# 잘못된 순서: 전체 데이터로 "정답과 관련 있어 보이는" 특성 20개를 고른 뒤 나눈다
골라 = SelectKBest(f_classif, k=20).fit_transform(X, y)
X_학습, X_시험, y_학습, y_시험 = train_test_split(골라, y, test_size=0.3, random_state=0)

# 올바른 순서: 먼저 나누고, 학습 데이터만 보고 고른다
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.3, random_state=0)
선택기 = SelectKBest(f_classif, k=20).fit(X_학습, y_학습)
```

```
누수 있음: 시험 점수 80%
누수 없음: 시험 점수 50%
```

fit은 학습 데이터로만, transform은 어디에나

2.3 정답을 품은 특성

```
# 해지할 때만 생기는 기록: 이탈한 고객에게만 위약금이 찍힌다
고객["위약금"] = np.where(고객["이탈"] == 1, rng.integers(1, 20, n) * 10000, 0)
```

```
가입개월·상담횟수만:  80.7%
위약금까지 넣으면:    100.0%
정답과의 상관계수:
가입개월    -0.18
상담횟수     0.19
위약금       0.87
```

100%는 예측이 아니라 **사후 확인**이다

너무 좋은 특성은 반가워하기 전에 의심부터

"예측하는 그 순간에 이 값을 **정말 알 수 있나?**"

2.3 같은 사람이 양쪽에, 그리고 누수 정리

```
줄 단위로 섞어 나누기: 양쪽에 다 나온 환자 33명
환자 단위로 나누기:    양쪽에 다 나온 환자 0명
```

누수	예	막는 법
전처리 통계	전체 평균으로 빈칸 채우기·스케일링	나눈 뒤 학습 데이터로만 fit
특성 선택	전체 데이터로 좋은 특성 고르기	나눈 뒤 학습 데이터로만 고르기
시험지 들여다보기	테스트 점수로 모델·설정 고르기	고르기는 검증으로, 테스트는 한 번
정답을 품은 특성	위약금, 처방된 약	예측 시점에 알 수 있는 값인지 확인
같은 대상의 여러 기록	한 환자의 여러 검사	환자 단위로 나누기 (GroupShuffleSplit)
중복 행	같은 줄이 학습·시험에 하나씩	나누기 전에 drop_duplicates()
미래 정보	내일 데이터로 어제를 예측	시간 순서로 자르기 (2일차)

직접 실행 10분

1. 02-data-preparation 폴더에서 b02_look.py, b04_peek.py, b05_leak_select.py를 실행한다
2. b04의 후보 모델 수 range(50)를 range(500)으로, 다시 range(5)로 바꾼다. 시험 점수와 새 데이터 점수는 어떻게 움직일까?
3. b05의 특성 수 5000을 50으로 줄이면(size=(100, 50)) "누수 있음" 점수는?

```
cd examples/vol2/02-data-preparation && uv run python b04_peek.py
```

다음 중 데이터 누수가 아닌 것은 무엇일까요?

가 전체 데이터의 평균으로 빈칸을 채운 뒤 나누기

나 이탈 예측 특성에 위약금 열 넣기

다 테스트 데이터를 맨 먼저 떼어 두고 끝까지 안 보기

라 한 환자의 검사 기록 네 줄을 줄 단위로 섞어 나누기

4차시 정리

데이터를 받으면 모델보다 먼저 **건강검진**: 빈칸, 숨은 빈칸, 표기 흔들림, 중복, 정답 비율

학습(문제집) · 검증(모의고사) · **테스트(수능)**. 테스트는 맨 먼저 떼고 마지막에 한 번

누수는 오류 없이 점수만 부풀린다. fit은 학습 데이터로만, transform은 어디에나

다음 차시: 재료 손질 · 결측치, 범주형 인코딩, 스케일링

5차시

05

결측치, 범주형 인코딩, 스케일링

책 2.4 ~ 2.6절

학습 목표

- 빈칸을 버릴지 채울지 정하고, 학습 데이터로 배운 값으로 채운다
- 순서 없는 범주는 원-핫으로, 순서 있는 범주는 순서를 적어 숫자로 바꾼다
- 스케일러 셋의 차이를 말하고, 시험 데이터를 학습 데이터의 자로 잰다

2.4 결측치: 버릴까, 채울까

```
# 열 다섯 개가 각자 10%씩 비어 있는 표 (예제용)
표 = pd.DataFrame(rng.normal(size=(n, 5)), columns=["가", "나", "다", "라", "마"])
for 열 in 표.columns:
    표.loc[rng.random(n) < 0.10, 열] = np.nan
```

```
열마다 빈 비율: [0.107, 0.106, 0.092, 0.101, 0.097]
한 칸이라도 빈 행 비율: 0.409
dropna() 후 남은 행: 591 / 1000
```

열마다 10%인데 **409줄**이 사라졌다

그래서 보통 **버리기보다 채우기**. 정답이 빈 행만 버린다

2.4 SimpleImputer: 학습 데이터로 배운 값으로

```
학습 = pd.DataFrame({"연소득": [3200, 4100, np.nan, 3800, 4500, 250000, np.nan, 3600]})
시험 = pd.DataFrame({"연소득": [np.nan, 5000]})

평균_채우기 = SimpleImputer(strategy="mean").fit(학습)
중앙값_채우기 = SimpleImputer(strategy="median").fit(학습)

표시 = SimpleImputer(strategy="median", add_indicator=True)
```

```
학습 데이터로 배운 평균: [44866.7]
학습 데이터로 배운 중앙값: [3950.]
시험 데이터 채우기(중앙값): [3950. 5000.]
      연소득  missingindicator_연소득
0  3950.0                    1.0
1  5000.0                    0.0
```

- 배운 값은 **statistics_**에, 시험에는 학습의 3950
- 튀는 값 하나에 평균이 끌려간다: 숫자 열은 **중앙값**
- add_indicator: 비어 있었다는 표시 열

2.4 비어 있다는 것도 정보다

```
소득 칸을 비운 비율: 24.9%
소득을 적은 사람의 연체율: 13.5%
소득을 비운 사람의 연체율: 30.9%
```

중앙값으로 채우기만 하면 이 **단서가 사라진다**. 표시 열로 지킨다

방법	언제	주의할 점
행 버리기	정답이 빈 행, 빈칸이 아주 드물 때	생각보다 많이 사라짐
열 버리기	열 대부분이 비었을 때	남은 값이 쓸모 있는지 먼저 확인
중앙값	숫자 열 기본값	튀는 값에 강함
평균	고르게 퍼진 숫자 열	튀는 값 하나에 끌려감
최빈값·"모름"	글자(범주) 열	"모름"은 하나의 범주가 됨
표시 열 추가	비어 있는 이유가 있을 법할 때	열이 하나씩 늘어남

2.5 원-핫 인코딩: 범주마다 스위치 하나

```
학습 = pd.DataFrame({"지역": ["서울", "부산", "대구", "서울", "부산"]})
시험 = pd.DataFrame({"지역": ["부산", "광주"]})      # 광주는 학습 때 못 본 값

인코더 = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
인코더.fit(학습)
print("새 열 이름:", 인코더.get_feature_names_out().tolist())
```

```
새 열 이름: ['지역_대구', '지역_부산', '지역_서울']
시험 데이터:
[[0. 1. 0.]
 [0. 0. 0.]]
```

못 본 광주는 스위치를 모두 끈다

pd.get_dummies를 따로 쓰면

```
학습: ['지역_대구', '지역_부산', '지역_서울']
시험: ['지역_광주', '지역_부산']
```

열이 달라진다: 배운 범주를 기억하지 않는다

5차시 · 2.5 범주형 인코딩 · 예제 02-data-preparation/b11_onehot.py (핵심 부분), b12_dummies_trap.py (출력)

2.5 순서가 있는 범주: 낮음 < 보통 < 높음

```
알아서 = OrdinalEncoder().fit(만족도)

순서대로 = OrdinalEncoder(categories=[["낮음", "보통", "높음"]],
                          handle_unknown="use_encoded_value", unknown_value=-1)
```

```
순서를 안 정하면: ['낮음', '높음', '보통']
[2. 1. 0. 2. 1.]
순서를 정해 주면: ['낮음', '보통', '높음']
[1. 2. 0. 1. 2.]
못 본 값: [-1.]
```

- 안 정하면 가나다순: 보통(2)이 높음(1)보다 커진다
- 순서는 **categories**에 직접 적는다
- 처음 보는 값은 **-1**로

2.5 종류가 너무 많은 범주: 드문 것은 묶는다

```
# 동네 이름 200가지가 섞인 주소 1000건 (예제용). 앞 번호일수록 흔하다
묶기 = OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=20).fit(동네)
```

```
서로 다른 동네: 163 곳
그냥 원-핫: 163 열
20번 미만은 묶기: 9 열 → ['동네_동000', '동네_동001', '동네_동002'] ... 동네_infrequent_sklearn
```

- 163곳이면 원-핫도 163열, 대부분 한두 번 나와 배울 거리가 없다
- 20번 미만인 동네는 **"드문 것들"** 한 칸으로 묶어 9열
- 처음 보는 동네도 그 칸으로: **infrequent_if_exist**

5차시 · 2.5 종류가 너무 많을 때 · 예제 02-data-preparation/b14_rare.py (핵심 부분)

2.6 스케일링: 눈금이 다르면 생기는 일

```
스케일러 = StandardScaler().fit(X_학습)      # 학습 데이터로만 평균·표준편차를 배운다
for 이름, 학습, 시험 in [("그대로", X_학습, X_시험),
                         ("표준화", 스케일러.transform(X_학습), 스케일러.transform(X_시험))]:
    모델 = LogisticRegression(max_iter=1000).fit(학습, y_학습)   # 최대 1000번까지 고쳐 본다
```

```
     mean area  mean smoothness
min      143.5          0.05263
max     2501.0          0.16340
그대로: 고친 횟수 1000번, 시험 정확도 93.7%
표준화: 고친 횟수   18번, 시험 정확도 95.8%
```

눈금이 고르면 **둥근 그릇**, 다르면 **길쭉한 협곡**: 지그재그로 헤맨다

ConvergenceWarning을 보면 반복 횟수보다 **스케일링부터**

5차시 · 2.6 눈금이 다르면 생기는 일 · 예제 02-data-preparation/b15_scale_gd.py (핵심 부분)

2.6 세 가지 스케일러

```
# 하루 방문자 수 (예제용). 마지막 날만 방송을 타서 크게 튀었다
방문자 = np.array([[120.0], [135.0], [128.0], [142.0], [131.0], [1500.0]])

StandardScaler  [-0.47 -0.44 -0.45 -0.43 -0.45  2.24]
MinMaxScaler    [0.   0.01 0.01 0.02 0.01 1.  ]
RobustScaler    [ -1.13   0.17  -0.43   0.78  -0.17 118.87]
```

스케일러	어떻게	언제
StandardScaler	평균을 빼고 표준편차로 나눈다	가장 무난한 기본 선택
MinMaxScaler	최솟값 0, 최댓값 1. 튀는 값에 나머지가 뭉친다	범위가 정해진 데이터(밝기 0~255)
RobustScaler	중앙값과 사분위 범위로 맞춘다	튀는 값이 많은 데이터

5차시 · 2.6 세 가지 스케일러 · 예제 02-data-preparation/b16_scalers.py (데이터 부분과 출력)

2.6 시험 데이터는 학습 데이터의 자로 잰다

```
학습 = np.array([[20.0], [35.0], [50.0], [65.0]])      # 학습 데이터의 나이
시험 = np.array([[30.0], [80.0]])                      # 시험 데이터의 나이

스케일러 = MinMaxScaler().fit(학습)                     # 최솟값 20, 최댓값 65를 배운다
잘못 = MinMaxScaler().fit(np.vstack([학습, 시험]))      # 시험 데이터까지 보고 배우면
```

```
배운 범위: [20.] ~ [65.]
학습: [0.    0.333 0.667 1.   ]
시험: [0.222 1.333]
```

1.333은 **맞는** 결과: 처음 보는 큰 값은 자의 끝을 넘는다

```
잘못 배운 범위: [20.] ~ [80.]
학습: [0.   0.25 0.5  0.75]
```

시험을 몰래 보고 만든 자: 0~1에 예쁘게 들어가도 틀린 방법

직접 실행 10분

1. b16_scalers.py에서 튀는 날 [1500.0]을 지우고 다시 실행한다. 세 스케일러 결과는 어떻게 달라질까?
2. b17_fit_train_only.py의 시험 나이 80.0을 10.0으로 바꾸면 시험 결과는?
3. b11_onehot.py에서 handle_unknown="ignore"를 지우고 실행하면 무슨 일이 생길까?

```
cd examples/vol2/02-data-preparation && uv run python b16_scalers.py
```

"지역" 열(서울, 부산, 대구)을 모델에 넣는 가장 알맞은 방법은?

Q

가 서울=1, 부산=2, 대구=3으로 번호 매기기

나 학습과 시험에 각각 pd.get_dummies 쓰기

다 학습 데이터로 OneHotEncoder를 fit하고, 못 본 값은 무시하게 하기

라 전체 데이터로 OneHotEncoder를 fit한 뒤 나누기

5차시 정리

결측치는 버리기보다 채우기. 숫자는 중앙값, 글자는 최빈값이나 "모름", 비운 이유가 있을 법하면 표시 열

범주는 순서가 없으면 원-핫(못 본 값은 ignore), 순서가 있으면 순서를 적어 OrdinalEncoder, 드문 것은 묶는다

스케일링은 기본이 표준화. 시험 데이터는 학습 데이터의 자로 잰다(0~1을 넘어도 맞다)

세 도구 모두: fit은 학습 데이터로만, transform은 어디에나

5차시 · 정리 · 복습: 책 2.4 ~ 2.6절

1일차에 배운 것

1. 규칙을 짜는 대신 **데이터와 정답**을 주고 규칙은 프로그램이 배운다
2. 최근접 이웃을 직접 짜고 **fit · predict · score**로 scikit-learn과 맞췄다
3. 목표는 **처음 보는 데이터**의 점수: 과적합, 눈금, 비교 기준
4. 먼저 살펴보고, 셋으로 나누고, **누수**를 막는다
5. 빈칸 채우기, 글자를 숫자로, 눈금 맞추기: **fit은 학습 데이터로만**

내일 2일차

- Pipeline · 불균형 · 시간 데이터와 이탈 실습
- 고전 머신러닝 모델들
- 평가와 교차 검증
- 하이퍼파라미터 탐색과 집값 종합 실습

1일차 마무리 · 복습: 책 1장 전체(1.12 제외), 2장 들어가며 ~ 2.6절